Proceedings of the 2004 IEEE
International Conference on Robotics & Automation
New Orleans, LA • April 2004

A New Sensor-Based Path-Planning Algorithm whose Path Length is Shorter on the Average

Hiroshi Noborio Ryo Nogami
Graduate School of Engineering
Osaka Electro-Communication University
Email: nobori@noblab.osakac.ac.jp

Satoru Hirao
Department of Engineering Informatics
Osaka Electro-Communication University
Hatsu-cho 18-8, Neyagawa, Osaka 572-8530, Japan

Abstract— **In many sensor-based path-planning algorithms, the worst path length has been theoretically evaluated, but the average path length will not be theoretically and experimentally evaluated at all. In general, shorting average path length is worth much than shorting worst path length in the practical use. Therefore, we propose a new sensor-based path-planning algorithm whose path length is exactly shorten on the average. The algorithm always expands the best crack of a spanned graph from an initial position. All cracks mean all tips of routes traced from hit and/or leave points and also all neighbors of hit and/or leave points not to trace. The evaluation and expansion work by A^* in the spanned graph. By selection of the best node, a mobile robot frequently reverses its direction following an uncertain obstacle. For this reason, the robot frequently passes through a shorter interval around an unknown obstacle three times or more. As a result, the worst path length cannot be bounded as any small value. However, a total driving distance is completely saved by following a short boundary three times or more, which is faster than following a long boundary two times. In this paper, we ascertain the superiority of our algorithm experimentally by a huge number of unknown environments.**

I. Introduction

In the last two decades, many researchers have proposed sensor-based path-planning algorithms while focusing on worst and average path lengths. Concerning to the worst path length, Sankaranarayanan classified and evaluated the lower bounds of two types of sensor-based path-planning algorithms. In the former type, a robot always goes round an obstacle as long as the robot encounters the obstacle. The lower bound is denoted by $D + 1.5\Sigma_i P_i$ (D: the Euclidean distance between initial and target positions, P_i: the perimeter of an i-th encountered obstacle) [1]. On the other hand, in the latter type, a robot does not always go round an obstacle even though the robot meets the obstacle. The lower bound is denoted by $D + 2\Sigma_i P_i$ [1]. As a result, the classic algorithm *Bug*1 proposed by Lumelsky is the best in the former type [2], and the algorithms *Alg*1 and *Alg*2 proposed by Sankaranarayanan are the best in the latter type [3],[4]. Recently, we propose the algorithms *Rev*1 and *Rev*2 whose worst path length is bounded by $D + 2\Sigma_i P_i$, and whose average path length is shorter than that of *Alg*1 and *Alg*2 [5],[6].

As contrasted with this, no researcher has evaluated the average path length in a general uncertain environment. As the unique exception, if each obstacle has simple shape such as circle in a general uncertain environment, we proposed a near-optimal sensor-based path-planning algorithm *Simple* and calculated a small competitive ratio between lengths of the optimal path and a selected path as 1.58 [7]. This requests a strong assumption of environment shape. In general, shorting average path length is more worth than shorting worst path length in the practical use. In addition, an algorithm whose worst path length is the shortest quite differs from another algorithm whose average path length is shorter. For example, *Bug*2 usually generates a shorter path than *Bug*1 on the average. The reason is that a robot always goes round an obstacle at least one time in *Bug*1 but the robot never follows a wide interval around the obstacle in *Bug*2.

In this paper, we propose a new sensor-based path-planning algorithm whose path length is shorter. For this purpose, we firstly revisit our classic algorithm $HD-I$ [8],[9]. In $HD-I$, a robot always compares clockwise and counter-clockwise following around a hit point of an encountered obstacle by a trial and error manner. Learning the direction escapes for a robot from joining a loop (a side trip to a destination) in an uncertain maze. To obtain a shorter path against $HD-I$, we propose a new algorithm *Ave*. *Ave* is regarded as a smart extension of $HD-I$ as follows: the local trial and error around a hit point can be extended to the global one in a spanned graph from an initial position. In *Ave*, a robot always compares all cracks of a spanned graph from an initial position to select a fruitful crack for getting a shorter path. First of all, after a robot hits an obstacle, the robot regards its clockwise and counter-clockwise neighbors as candidates of cracks. Secondly, when a robot traces an encountered obstacle by the clockwise or counter-clockwise direction, its tip point is a candidate of crack. Finally, after a robot leaves an obstacle, its clockwise or counter-clockwise point not to trace is defined as a candidate of crack. In addition, if a robot encounters a visited position and consequently joins a loop, the robot also seeks for a fruitful crack in a spanned graph in *Ave*. The algorithm A^* always picks up the fruitful crack in the spanned graph [10].

In conclusion, we use the local trial and error around an encountered obstacle in $HD-I$. As contrasted with this, we use the global trial and error on a spanned graph in *Ave*. The most fruitful crack can be evaluated by two distances. One is the distance on the shortest route from the present position to the crack. The path is selected by A^* on a graph spanned from an initial position. Another is the Euclidean distance from the

0-7803-8232-3/04/$17.00 ©2004 IEEE

crack to a destination. This is an underestimation between their positions. While selecting such fruitful cracks successively, a robot arrives at the destination earlier on the average.

Finally, we explain some relationships against many kinds of sensor-based path-planning algorithms with wide views. They have been recently proposed [11],[12],[13],[14]. First of all, VisBug uses the range data only to find shortcuts relative to the path generated by *Bug*2. In succession, TangentBug is a new version of VisBug that specifically exploits range data. The algorithm uses the notion of a tangent graph to construct a local one, which is used to produce paths that often resemble the shortest path to the goal. In TangentBug, we assume that a mobile robot senses surrounding obstacles omnidirectionally. However, some of TangentBug's assumptions do not apply to the **rover problem** of navigating in planetary terrain. For example, TangentBug assumes that obstacles block both moving and sensing, and does that the robot's sensor provides an omnidirectional view. To overcome these, WedgeBug and RoverBug have been proposed.

Roughly speaking, ideas in our algorithm can be straightforwardly mixed with the above algorithms assuming wider sensing areas. All the above algorithms do not consider any trial and error front of each uncertain obstacle. They only consider how to short an optimal path locally based on sensing information. If and only if the sensor range is infinite, we can pick up the globally optimal path by using the tangential graph among known obstacles [15]. Unfortunately, we do not have such a powerful sensor and consequently sensor range is strictly limited in our living space, which differs from planetary terrain. For this reason, we do not always use any technique in these algorithms and also compare all sensor-based path-planning algorithms in a complicated maze whose obstacle completely blocks moving and sensing.

The paper is organized as follows: Section 2 describes a robot and its unknown environment. In section 3, we explain the classic best algorithm *HD–I* and a proposed new algorithm *Ave*. In section 4, we compare *Rev*1, *Rev*2, $HD-I$ and *Ave* concerning to the average path length for an enormous number of 2-D uncertain mazes. Finally, section 5 presents a few concluding remarks.

II. Classic and Proposed Sensor-Based Path-Planning Algorithms

In this section, we explain details of classic better algorithm $HD-I$ and new best algorithm *Ave*.

A. Mobile Robot and Its Uncertain Environment

In this paragraph, we define our mobile robot and its unknown environment. In this paper, we prepare a huge number of 2-D environments. Each environment has many obstacles whose number is of finite and its perimeter is also bounded. Therefore, a robot never continues to follow an obstacle.

On the other hand, a robot is defined as a point, and consequently can pass among many obstacles. A robot does not know any shape and location of each obstacle in a 2-D environment. A robot always identifies its present position by GPS (Global Positioning System), and also knows its destination in advance. The robot can move omni directionally and can recognize an unknown obstacle omni directionally. By the sensor-feedback, the robot faithfully traces around an encountered obstacle. Finally, the robot can memorize a lot of positions and their distances from an initial position as a spanned graph.

B. Classic Algorithm HD – I

In this section, we explain detail of *HD–I* in a 2-D uncertain environment. A robot supervised by *Rev*1 and *Rev*2 traces obstacle boundary at most two times but following boundary is too long. Compared with this, a robot controlled by $HD-I$ traces obstacle boundary three times or more but the boundary is too short (Fig.1). As a result, $HD-I$ selects a shorter path on the average. We consider two types of $HD-I$: $HD-I/wo$ does not include the segment condition and $HD-I/w$ includes the segment condition explained later. The algorithm always compares two tips of clockwise and counter-clockwise routes spanned from the last hit point (e.g., H_1, H_2 and H_3) in order to avoid an encountered obstacle earlier.

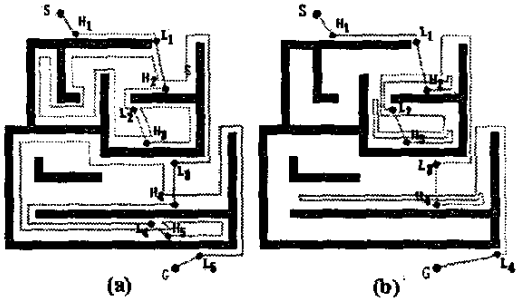

Fig. 1. (a) A robot follows each interval at most two times in *Rev*2. (b) A robot traces each interval three times or more in $HD-I/wo$.

1. Set $i = 1$, $R = L_{i-1} = S$.

2. From a leave point L_{i-1}, a robot R goes straight to a destination G until the following occurs:

(2a) If a robot R arrives at a goal G, exit with success.

(2b) If a robot R encounters an uncertain obstacle at a hit point H_i, the robot regards clockwise and counter-clockwise neighbor points as new tips. Then, the robot always compares two tips to select the better one. Two tips are always spanned from the last hit point H_i.

The detail of comparison is as follows: The robot is basically on one of two tips of routes from H_i. The point is denoted as a present point P_p. If the distance h_p of straight segment between P_p and G is larger than the sum of two distances explained later, the robot switches following direction to move from a present tip P_p to its opposite tip P_o. Otherwise, the robot continues to trace the obstacle. Two distances are as follows: The former is a driving distance from a present tip P_p to its opposite tip P_o, which is evaluated as $\alpha \times m$ (α: an arbitrary coefficient is set as 0.5, m: distance of the path between two tips.). The latter is the distance h_o of straight segment between P_o and G.

(2c) If a robot R is closer to a goal point G than a closest point C, it is replaced by R. Also, if the direction toward G is opened in $HD-I/wo$, R leaves the obstacle at a leave point L_i. In addition, if the robot is located on the straight segment between S and G in $HD-I/w$, R also leaves the obstacle at L_i. This is called as **segment condition**.

(2d) If a robot returns to the last hit point H_i without finding L_i, exit with failure. In this case, a goal point G is completely enclosed by obstacle boundary and therefore there is no deadlock-free path until the goal point G.

C. *New Algorithm* Ave

In this section, we explain detail of *Ave* in a 2-D uncertain environment. A robot supervised by $HD-I$ always compares only two tips expanded from the last hit point. Compared with this, a robot controlled by *Ave* always compares all cracks in a spanned graph from an initial position (Fig.2). The set of cracks is as follows: Firstly, after a robot hits an obstacle at H_i, its clockwise and counter-clockwise neighbors of H_i are as candidates of cracks. Secondly, if a robot traces the obstacle, the robot always renews one neighbor as a candidate of crack. Finally, after a robot leaves an obstacle at L_i, its clockwise or counter-clockwise neighbor of L_i not to trace is defined as a candidate of crack. As a result, *Ave* selects a shorter path on the average. We consider four types of *Ave* as *Ave/wo*, *Ave/w*, *Ave/lo* and *Ave/l*: *Ave/wo* and *Ave/lo* do not include the segment condition explained later. *Ave/w* and *Ave/l* include the segment condition. *Ave/wo* and *Ave/w* consider only cracks spanned from all the hit points. *Ave/lo* and *Ave/l* consider cracks spanned from all the hit and leave points.

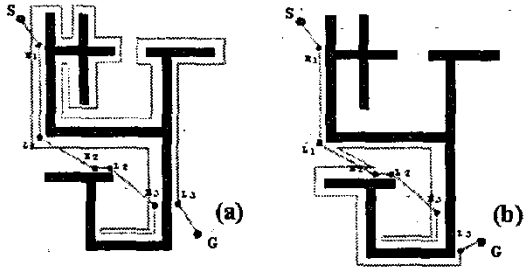

Fig. 2. (a) An path selected by $HD-I/wo$ whose $\alpha = 0.5$ in an uncertain maze. (b) An path selected by *Ave/wo* whose $\alpha = 0.5$ in an uncertain maze.

In every *Ave*, we succeed in shorting average path length while searching a graph spanned from an initial position by A^*. The reason to select the shortest route in the graph by A^* is as follows: there are three or more routes from a present position to each candidate of all cracks. An example shown in Fig.3 has three routes from the present position R to a crack $R(H_4)$. Finally in four types of *Ave*, a robot R traces boundary of an obstacle and renews its neighbor crack in **simple avoidance**, or moves to another non-neighbor crack whose f is minimum in **complex avoidance** without entering into any loop, or returns to another non-neighbor crack whose f is minimum in **loop avoidance** after joining into a loop.

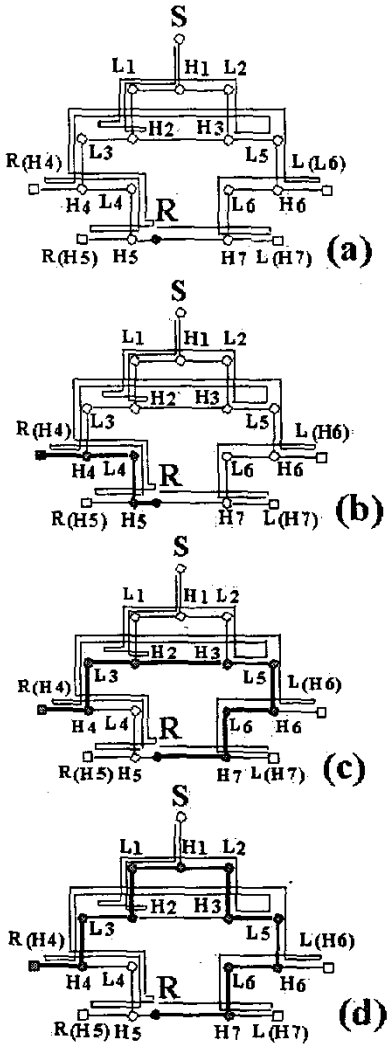

Fig. 3. (a) A present graph spanned from the initial position. (b) First path from R to $R(H_4)$. (c) Second path from R to $R(H_4)$. (d) Third path from R to $R(H_4)$.

1) Algorithm **Ave/wo**: First of all, we initialize a start position as S, a goal position as G, a present position as R, and the minimum Euclidean distance to the destination as *Dist*. Then, we successively construct each hit position as $H_i(i = 1, 2, \cdots)$, clockwise and counter-clockwise cracks as $L(H_i)$ and $R(H_i)$, each leave position as $L_i(i = 1, 2, \cdots)$, and a list of unexpanded cracks as *Open*.

Secondly, in order to sort all unexpanded cracks in *Open*, we calculate an evaluation value f of each crack as $f = \alpha \times g + h$ (α is a coefficient arbitrarily selected between 0.0 and 1.0, g: Euclidean distance of the shortest route from a present position to each crack, h: Euclidean distance of the straight segment between each crack and the destination). The shortest route is selected by the algorithm A^* on a graph spanned from a start position.

1. Set $i = 1$, $L_{i-1} = R = S$.

2. From the leave point L_{i-1}, a robot R goes straight to a destination G until the following occurs:

(2a) If a robot R arrives at a goal G, exit with success.

(2b) If a robot R encounters an uncertain obstacle at a hit point H_i, the robot sets $H_i = R$, $Dist = |RG|$, and then memorizes clockwise and counter-clockwise cracks $L(H_i)$ and $R(H_i)$ neighboring H_i in *Open*. Then, we sort all cracks in the order of heuristics f. Then, we move to the step 3.

3. A robot R always selects the top crack $R(H_m)$ or $L(H_m)$ (whose heuristics f is the smallest) in *Open*, and then eliminates $R(H_m)$ or $L(H_m)$ from *Open*. In succession, we construct $R(H_m)$ or $L(H_m)$ neighboring the eliminated crack as unvisited crack, and compare the constructed crack with the top crack in *Open*.

Simple avoidance: If f of the constructed crack is smaller than f of the top crack, g (and consequently f) of every crack in *Open* increments by the same driving distance. Since the order of h is invariant, without sorting all elements in *Open*, a robot moves to the constructed crack, builds its neighbor not to trace as a candidate of crack, and memorizes the crack into *Open*. Finally, this step is continued.

Complex avoidance: If f of the top crack is smaller than f of the constructed crack, a robot moves to the top crack, whose route is the shortest path selected by A^* in a graph spanned from a start position. Then, we build its neighbor not to trace as a candidate of crack, and memorizes the crack into *Open*. In succession, we select shortest paths from the present crack to all cracks stored in *Open* by A^*, and get their g on their paths, and renew their f in order to sort all cracks. Finally, this step is continued.

This procedure is continued until the following occurs:

(3a) If a robot R arrives at a goal G, exit with success.

(3b) If the Euclidean distance $|RG|$ is smaller than *Dist*, we set *Dist* as $|RG|$. Furthermore, if the direction toward G is not interrupted by any obstacle, we set L_i as R, eliminate the top crack in *Open*, set $i = i + 1$, and then backs to the step 2.

(3c) If R goes round an obstacle, that is, $R = R(H_i)$ or $R = L(H_i)$ without finding L_i, we understand a robot cannot arrive at G. For this reason, exit with failure.

(3d) Loop avoidance: First of all, if R arrives at $R(H_k)$ or $L(H_k)$ in *Open*, we eliminate the crack (and also the same one $L(H_i)$ or $R(H_i)$) in *Open*. Secondly, we select the shortest paths from the present crack to all cracks stored in *Open* by A^*, and get their g on their paths, and renew their f in order to sort all cracks in *Open*. Thirdly, a robot R moves to the top crack in *Open*. Fourthly, we select the shortest paths from the present crack to all cracks stored in *Open* by A^*, and get their g on their paths, and renew their f in order to sort all cracks in *Open*. Finally, this step is continued.

(3e) If a robot R revisits a previous leave point L_k again, we eliminate all candidate cracks whose integer numbers n are satisfied in the inequality $k \leq n \leq i$ and whose types are the same against the crack selected last in *Open* (e.g., if the last crack is $R(H_i)$ or $L(H_i)$, we eliminate all cracks $R(H_n)$ or $L(H_n)$). Then, we use **loop avoidance**, and finally this step is continued.

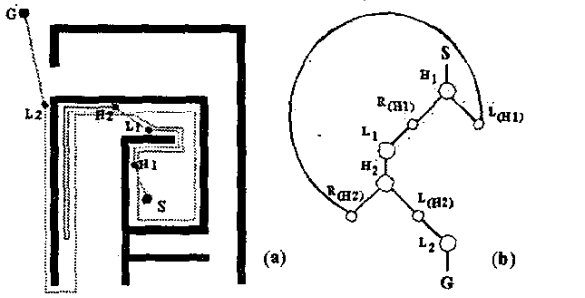

Fig. 4. (a) A path selected by *Ave/wo* in a maze. (b) A robot reaches its destination G in a final graph.

In an unknown maze illustrated in Fig.4(a), we explain some behaviors in *Ave/wo* ($\alpha = 0.5$). First of all, a robot R goes straight from S to G, and encounters an obstacle at a hit point H_1. R constructs clockwise and counter-clockwise cracks $L(H_1)$ and $R(H_1)$ neighboring H_1, and memorizes two candidate cracks into *Open*. Then, we sort all candidates, i.e., $R(H_1)$ and $L(H_1)$ by their heuristics f in *Open*. As mentioned previously, f consists of g and h, which are measured by the Euclidean distances.

Moreover, h is defined as the Euclidean distance from each crack $R(H_1)$ or $L(H_1)$ to G. Therefore, we calculate $f(R(H_1)) = 0.5 \times 1 + |R(H_1)G|$, $f(L(H_1)) = 0.5 \times 1 + |L(H_1)G|$. Since $R(H_1)$ is the top crack of *Open* in this environment, i.e., $f(R(H_1)) < f(L(H_1))$, a robot R moves to the crack $R(H_1)$ and we eliminate $R(H_1)$ from *Open*. Secondly, a robot R builds an unvisited crack $R(H_1)$ neighboring the eliminated crack, and then compares f of the generated crack $R(H_1)$ with f of the top crack $L(H_1)$ in *Open*. In this case, since f of the generated crack $R(H_1)$ is smaller, $R(H_1)$ is stored as the top crack in *Open* by **simple avoidance**. $R(H_1)$ is always replaced of its neighbor not to trace. Here, we need not sort all candidates in *Open* because the order of f is invariant. By using **simple avoidance** successively, R finally arrives at L_1. Then, R leaves the obstacle from L_1 to go straight to G. Moreover, R hits an obstacle at H_2, constructs clockwise and counter-clockwise cracks (i.e., $L(H_2)$ and $R(H_2)$), and stores them into *Open*. Then, we sort all cracks in the order of f. Then, a robot successively renews $L(H_2)$ by **simple avoidance**.

After a while, f of the crack $R(H_2)$ in *Open* is smaller than f of a renewed crack $L(H_2)$. In this case, we use **complex avoidance**. First of all, we select the shortest route to $R(H_2)$ by A^*. The shortest route until $R(H_2)$ is denoted as $R \sim H_2R(H_2)$ (R: present robot position, $\sim$: tracing obstacle boundary). A robot arrives at $R(H_2)$ via the route. Since $L(H_2)$ is still left in *Open*, the route until $L(H_2)$ is stored in a spanned graph. Here, graphs before and after reversing the direction are denoted in Fig.5(a),(b), respectively. Then, we select shortest paths from the present position to candidates $R(H_2)$, $L(H_2)$, $L(H_1)$ by A^*, change their d and then f, and consequently sort them in the order of f. Finally, since R successively renews $R(H_2)$ in *Open* by **simple avoidance**, R continues to trace an obstacle.

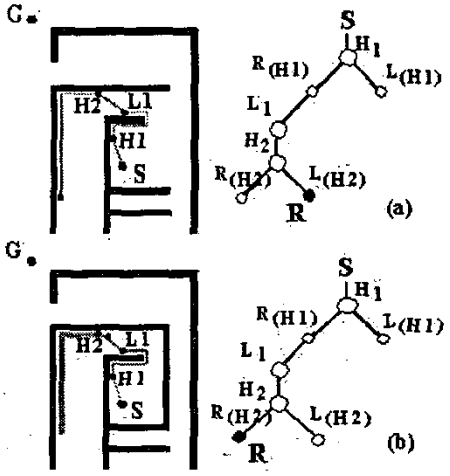

Fig. 5. **Complex avoidance:** (a) A path and a graph before **complex avoidance**. (b) A path and a graph after **complex avoidance**.

Now, if R arrives at $L(H_1)$ in *Open*, we use **loop avoidance** as follows: Firstly, we eliminate $R(H_2)$ and $L(H_1)$ from *Open*. Therefore, we search the shortest path from the present position R to only the crack $L(H_2)$ in *Open* by A^*. At present, we describe a path and a graph in Fig.6(a),(b), respectively. There are two routes in Fig.6(c),(d). The route $R \sim L_1 - H_2 \sim L(H_2)$ (R: present robot position, $-$: going straight to the destination, $\sim$: avoiding obstacle boundary) (Fig.6(c)) is selected as the shortest route and therefore a robot moves to $L(H_2)$ via the route. Then, we select the shortest path to only the crack $L(H_2)$ by A^*, change its d and then f, and consequently sort them in the order of f in *Open*.

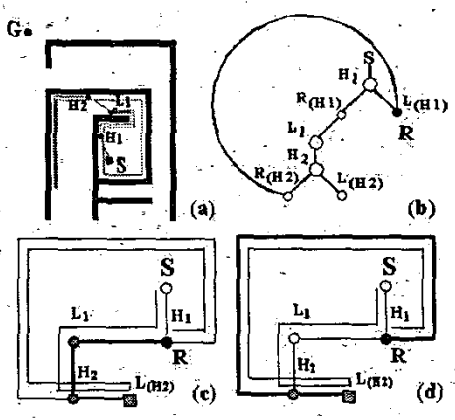

Fig. 6. **Loop avoidance:** (a) A path with a loop. (b) A graph with a loop. (c) One route from R to $L(H_2)$. (d) Another route from R to $L(H_2)$.

Moreover, a robot R successively traces an obstacle by **simple avoidance**. Then, a robot R leaves an obstacle from L_2, and finally arrives at G. The final spanned graph is illustrated in Fig.4(b).

2) Algorithm **Ave/w:** In this paragraph, we explain the algorithm *Ave/w*. This is designed by changing **(3b)** of *Ave/wo* slightly.

(3b) If R is on SG and also $|RG|$ is smaller than *Dist*, we set $Dist = |RG|$. Moreover, if the direction toward G is not interfered by any obstacle, we set a leave point L_i by R, we eliminate the top crack in *Open*, we set $i = i + 1$, and go back to the step 2.

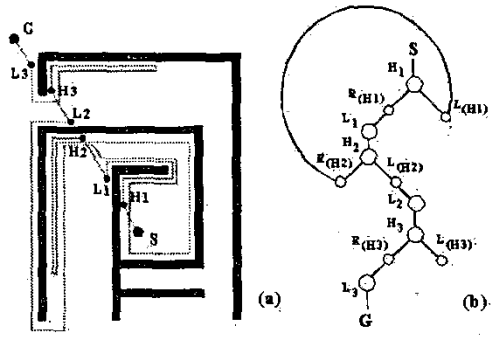

Fig. 7. (a) A path selected by *Ave/w* in a maze. (b) A robot reaches its destination G in a final graph.

3) Algorithms **Ave/lo** *and* **Ave/l:** In this paragraph, we explain new algorithms *Ave/lo* and *Ave/l*. They are designed by changing **(3b)** and **(3e)** of *Ave/wo* and *Ave/w* slightly. The unique difference is to regard $L(L_i)$ and $R(L_i)$ as clockwise and counter-clockwise cracks neighboring L_i. The others are the same.

(3b) If the Euclidean distance $|RG|$ is smaller than *Dist* in *Ave/lo* (in addition to this, R is on the segment SG in *Ave/l*), we set *Dist* as $|RG|$. Furthermore, if the direction toward G is not interrupted by any obstacle, we set L_i as R, we eliminate the crack selected last in *Open*. If following direction is right (counter-clockwise) [left (clockwise)], we store a candidate crack $R(L_i)$ [$L(L_i)$] into *Open*. Then, we set $i = i + 1$ and return to the step 2.

(3e) If a robot R revisits a candidate crack $R(L_k)$ or $L(L_k)$, we firstly eliminate the crack (and also $L(H_i)$ or $R(H_i)$). Then, we eliminate two sets of cracks whose integer numbers n are satisfied in the inequality $k \le n \le i$. The former set covers cracks neighboring hit points H_n, whose types are the same against the crack selected last. The latter set covers cracks neighboring leave points L_n, whose types are opposite from the selected crack in *Open*. For example, if the selected crack is $R(H_i)$ or $L(H_i)$ ($L(L_k)$ or $R(L_k)$), we eliminate all candidates $R(H_n)$ and $L(L_n)$ or $L(H_n)$ and $R(L_n)$ ($k \le n \le i$). Then, we use **loop avoidance**, and finally this step is continued.

After tracing obstacle boundary from a crack $R(H_i)$ [$L(H_i)$] neighboring a hit point H_i, a robot R sometimes meets another crack $L(L_j)$ [$R(L_j)$] (i $\le$ j) neighboring a leave point L_j (Fig.8(b)). The reason is as follows: when a robot stands behind an obstacle, the robot should avoid the obstacle by left or right direction. The left (clockwise) following boundary of an obstacle always encounters the right (counter-clockwise) following the boundary.

III. Simulation Results

In this section, we ascertain our algorithm *Ave* is the best concerning to average path length among all kinds of sensor-

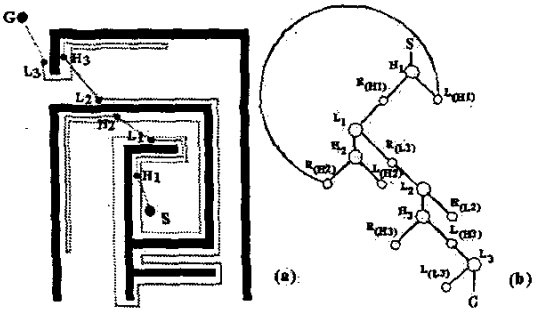

Fig. 8. (a) A path selected by *Ave/lo* in a maze. (b) A robot reaches its destination G in a final graph.

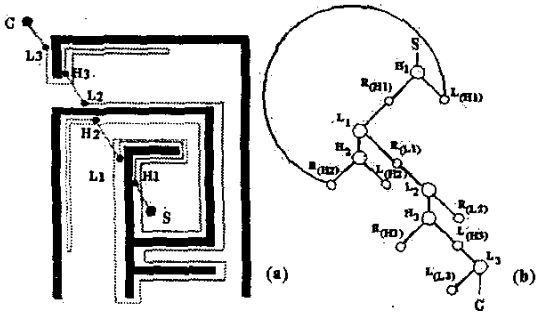

Fig. 9. (a) A path selected by *Ave/l* in a maze. (b) A robot reaches its destination G in a final graph.

based path-planning algorithms. To ascertain this, we firstly check our classic algorithms $HD-I/wo$ and $HD-I/w$ are better than the best classic algorithms *Rev*1 and *Rev*2. Secondly, we investigate new algorithms *Ave/wo*, *Ave/w*, *Ave/lo* and *Ave/l* are better than $HD-I/wo$ and $HD-I/w$ in two sets of uncertain mazes generated randomly.

A. How to Construct Unknown Mazes Randomly

First of all, we explain how to design a huge number of mazes randomly. Each maze consists of $100 = 10 \times 10$ cells (Fig.10). Each cell neighbors four walls and four pillars. If at least one wall is located, its neighboring two pillars are also located. In other words, no pillar is located if its neighbor four walls are not located. To construct each maze flexibly, we use a set of digitalized cells. For example, in order to describe a maze with $100 = 10 \times 10$ cells, we prepare 76×76 pixels. 76 means 5 (cell including inner passage) $\times$ 10 + 2 (wall and pillar) $\times$ 11 + 2 (outer passage) $\times$ 2. That is, each cell consists of $25 = 5 \times 5$ pixels, each wall consists of $10 = 5 \times 2$ pixels, and each pillar consists of $4 = 2 \times 2$ pixels. Finally, we have a passage whose width is two pixels, which should be kept around each wall. In conclusion, there are 220 walls and 121 pillars in one maze (Fig.10). A random function based on a clock time of PC determines whether each wall appears or not. The appearance percentage is about 60.

We prepare two sets of 10×10 mazes. In the former set, an uncertain environment is not surrounded at all by a series of walls. As contrasted with this, in the latter set, it is surrounded by the series, and consequently passages outside mazes are not available and robot behaviors are restricted a little. As only the exception, if and only if there is at least one closed area in a maze (Fig.11), there is no path for some pairs of start and goal positions in the maze. For this reason, we should eliminate such a maze. If we adopt mazes smaller than 10×10, they frequently include at least one closed region not to be used for the evaluation.

After getting a maze, we combinational select a pair of start and goal points whose are allocated at centers of cells, and we totally calculate the sum of path lengths for all pairs. The number of centers is 100, and therefore the number of pairs is $9900 = 100 \times 99$. Each path length is calculated by summing distances between all neighbor pixels. The distance between horizontal and vertical neighbor pixels is defined by 1, and the distance between diagonal neighbor pixels is defined by $\sqrt{2}$.

By using the wall and pillar sets, we can design an enormous number of mazes in order to evaluate classic and new sensor-based path-planning algorithms. The reason is because a robot repeatedly enters into two kinds of loops including and excluding the destination G in such a complex maze. Even in such a size, the number of total mazes is beyond $2^{220} \geq 1.6849 \times 10^{66}$. Therefore, it is difficult for us to construct all mazes. On the observation, if and only if superiority between these algorithms is invariable per 100 mazes, we judge the superiority is true.

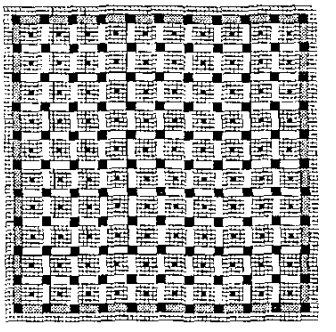

Fig. 10. A digitalized image including cells, walls, pillars and passages. A series of gray walls are to be the surrounding.

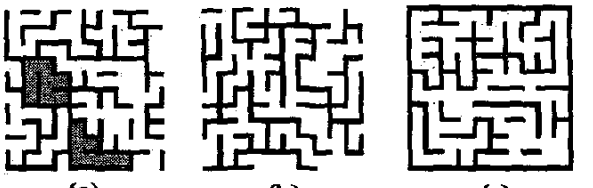

Fig. 11. (a) A maze including two closed regions. (b) A maze without the outer frame. (c) A maze with the outer frame.

First of all, we compare all algorithms in 10×10 with and without the outer frame and describe their total path lengths in Tables I and II. The algorithms are *Rev*1, *Rev*2, $HD-I$ ($\alpha = 0.5$) and *Ave* ($\alpha = 0.5$). Comparing the best classic algorithms *Rev* and $HD-I$ on average path length, we understand $HD-I$ is better than *Rev*. The reason is as follows: The number following an interval around an obstacle is bounded by at most two in *Rev*, on the other hand, the number is not limited in

TABLE I

THE SUM OF LENGTHS FOR ALL PATHS MADE BY *Rev*1, *Rev*2, *HD − I/w*, *Ave/w*, *Ave/l*, *HD − I/wo*, *Ave/wo* AND *Ave/lo* IN AN ENORMOUS NUMBER OF 10 × 10 UNCERTAIN MAZES WITHOUT ANY SURROUND.

number of mazes	*Rev*1	*Rev*2	*HD − I /w*	*Ave/w*	*Ave/l*	*HD − I /wo*	*Ave/wo*	*Ave/lo*
0-99	3760753	3170216	2901139	2810020	2884225	2466055	2264082	2374625
100-199	3679462	3072452	2824585	2717495	2712399	2392089	2202660	2208986
200-299	3878689	3301216	3024625	2921230	2859359	2578695	2376336	2384306
300-399	3707482	3145570	2858988	2825778	2772584	2412950	2244346	2252068
400-499	3905351	3309644	3015768	2954947	2963075	2570221	2344614	2354141
500-599	3819841	3211856	3010402	2914046	2825483	2464117	2257939	2359564
600-699	3904188	3304797	3143813	3050423	2728545	2598320	2373631	2216401
700-799	3973887	3339205	3103125	3078177	3047115	2644299	2421935	2430872
800-899	3623596	3022380	2786521	2731783	2758777	2348241	2161167	2167012
900-999	3022380	3193118	2946972	2937426	2926830	2507595	2305040	2312104

TABLE II

THE SUM OF LENGTHS FOR ALL PATHS MADE BY *Rev*1, *Rev*2, *HD − I/w*, *Ave/w*, *Ave/l*, *HD − I/wo*, *Ave/wo* AND *Ave/lo* IN AN ENORMOUS NUMBER OF 10 × 10 UNCERTAIN MAZES SURROUNDED BY A SERIES OF WALLS.

number of mazes	*Rev*1	*Rev*2	*HD − I /w*	*Ave/w*	*Ave/l*	*HD − I /wo*	*Ave/wo*	*Ave/lo*
0-99	4033546	2957181	2706036	2524259	2521108	2126184	1730137	1763767
100-199	4084576	2990692	2726453	2462383	2471377	2119491	1747891	1780669
200-299	4139682	3064786	2768157	2517915	2528174	2195123	1755664	1786978
300-399	4154965	3079838	2795164	2682545	2709066	2202895	1777890	1808562
400-499	4131099	3081734	2812792	2521192	2547211	2233449	1804537	1836720
500-599	4242449	3159792	2906308	2697129	2722114	2318328	1849298	1883649
600-699	4184382	3099774	2836487	2534787	2589111	2239497	1818606	1850054
700-799	4152700	3081826	2822887	2583021	2660398	2205553	1816640	1847267
800-899	4048809	2970237	2695239	2355477	2413474	2121616	1751368	1778398
900-999	4150972	3063767	2808857	2478359	2541093	2214267	1794646	1825983

HD − I. By no restriction, a robot freely reverses its direction following an obstacle and frequently escapes from a loop in *HD − I*. Therefore, the number of loops in a path selected by *HD − I* is smaller than the number of loops in a path selected by *Rev*. As a result, a path with shorter intervals traced many times is better than that with longer intervals avoided two times. This comparative example is described in Fig.1

Secondly, we compare the best classic algorithm *HD−I* and our new algorithm *Ave* on average path length. The average path length of *Ave* is shorter than that of *HD−I*. The reason is as follows: a robot compares clockwise and counter-clockwise cracks around a following obstacle in *HD − I*. Therefore, depending on α, a robot cannot return to the opposite crack because its distance from the present crack is too long. To overcome this drawback, a robot compares all cracks on a graph spanned from an initial position in *Ave*. Returning the best crack via the shortest path selected by A^*, a robot finally picks up a shorter path until the destination *G*. This comparative example is described in Fig.2.

Finally, when the parameter α of heuristics is flexibly altered, we compare four types of algorithms *Ave* and describe their total path lengths in Tables III and IV. Although many other values are tested as α, we obtain the smallest sum in $\alpha = 0.25$. Overall, if α is too small, a robot goes and backs the same interval many times, if α is too large, a robot never backs and enters into a loop. An example is shown in Fig.12.

Finally, we compare *Ave/wo* or *Ave/w* with *Ave/lo* or *Ave/l*. *Ave/lo* or *Ave/l* has all cracks generated from hit and leave points, on the other hand, *Ave/wo* or *Ave/w* has a part of the cracks, which are made from hit points. Nevertheless, average path length of *Ave/wo* or *Ave/w* is better than that of *Ave/lo*

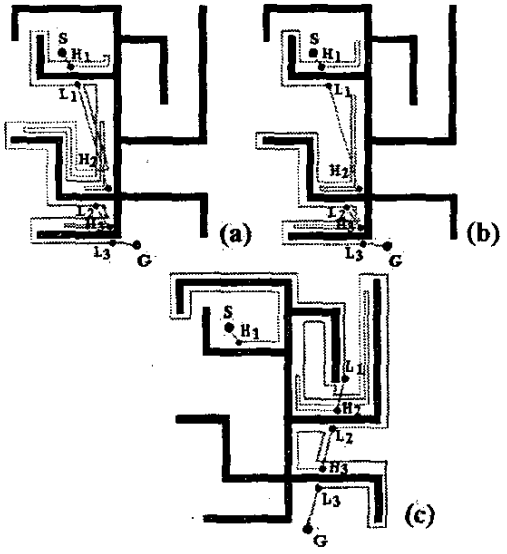

Fig. 12. (a) A path selected in *Ave/wo* whose $\alpha = 0.15$ in a uncertain maze. (b) A path selected in *Ave/wo* whose $\alpha = 0.25$ in the maze. (c) A path selected in *Ave/wo* whose $\alpha = 0.5$ in the maze.

or *Ave/l* (Tables I and II, Fig.13). This result shows us that path length is not always depend on the number of candidate cracks. In other words, since a robot sometimes wonders many possibilities, the robot selects a longer path.

IV. CONCLUSIONS

In this paper, concerning to average path length, we compare all the sensor-based path-planning algorithms with each other, and then we regard the algorithm *Ave* as the best. The reason

TABLE III

IF α IS SET AS 0.15, 0.25 AND 0.5, RESPECTIVELY, THE SUM OF LENGTHS IS CALCULATED FOR ALL PATHS MADE BY *Ave/w*, *Ave/l*, *Ave/wo* AND *Ave/lo* IN AN ENORMOUS NUMBER OF 10×10 UNCERTAIN MAZES WITHOUT ANY SURROUND

	$\alpha = 0.15$				$\alpha = 0.25$				$\alpha = 0.5$			
number of mazes	*Ave/w*	*Ave/l*	*Ave/wo*	*Ave/lo*	*Ave/w*	*Ave/l*	*Ave/wo*	*Ave/lo*	*Ave/w*	*Ave/l*	*Ave/wo*	*Ave/lo*
0-99	2710478	2833275	2122321	2332782	2655437	2766614	2013202	2271360	2810020	2884225	2264082	2374625
100-199	2597933	2804508	2102076	2282619	2532026	2681172	2048765	2108122	2717495	2712399	2202660	2208986
200-299	2799311	2931157	2202399	2406731	2731797	2899432	2100365	2302472	2921230	2859359	2376336	2384306
300-399	2638658	2886266	2158622	2358771	2571079	2703166	2079786	2270012	2825778	2772584	2244346	2252068
400-499	2713831	3100694	2368922	2390803	2632713	2896654	2287324	2296795	2954947	2963075	2344614	2354141
500-599	2618157	3103958	2270607	2312028	2559696	2899498	2195477	2200980	2914046	2825483	2257939	2359564
600-699	2760801	3112168	2172203	2426313	2685661	2900825	2109521	2322389	3050423	2728545	2373631	2216401
700-799	2784012	3116305	2213585	2488174	2724237	2933750	2147170	2400222	3078177	3047115	2421935	2430872
800-899	2509624	2883058	2121100	2209710	2465861	2718879	2030202	2105229	2731783	2758777	2161167	2167012
900-999	2760137	3066455	2203361	2403499	2629369	2899133	2129823	2337671	2937426	2926830	2305040	2312104

TABLE IV

IF α IS SET AS 0.15, 0.25 AND 0.5, RESPECTIVELY, THE SUM OF LENGTHS IS CALCULATED FOR ALL PATHS MADE BY *Ave/w*, *Ave/l*, *Ave/wo* AND *Ave/lo* IN AN ENORMOUS NUMBER OF 10×10 UNCERTAIN MAZES SURROUNDED BY A SERIES OF WALLS.

	$\alpha = 0.15$				$\alpha = 0.25$				$\alpha = 0.5$			
number of mazes	*Ave/w*	*Ave/l*	*Ave/wo*	*Ave/lo*	*Ave/w*	*Ave/l*	*Ave/wo*	*Ave/lo*	*Ave/w*	*Ave/l*	*Ave/wo*	*Ave/lo*
0-99	2265323	2587566	1785793	1832951	2184358	2573381	1622310	1702332	2524259	2521108	1730137	1763767
100-199	2238186	2561367	1708104	1895678	2156113	2400218	1618328	1835580	2462383	2471377	1747891	1780669
200-299	2260666	2678450	1700251	1868477	2195135	2488972	1631922	1844012	2517915	2528174	1755664	1786978
300-399	2300205	2774885	1754434	1870731	2243050	2683322	1647465	1763311	2682545	2709066	1777890	1808562
400-499	2334973	2638248	1814295	1921926	2293317	2494466	1676309	1770013	2521192	2547211	1804537	1836720
500-599	2385994	2758513	1769135	1974339	2312155	2768752	1704882	1829551	2697129	2722114	1849298	1883649
600-699	2405840	2637997	1746019	1948221	2316116	2553982	1691597	1798167	2534787	2589111	1818606	1850054
700-799	2364394	2719707	1801382	1953460	2301437	2620879	1678191	1779083	2583021	2660398	1816639	1847267
800-899	2194394	2535987	1701382	1851841	2133145	2400095	1628694	1759926	2355477	2413474	1751368	1778398
900-999	2367464	2603174	1726540	1894733	2271806	2588567	1653782	1803348	2478359	2541093	1794646	1825983

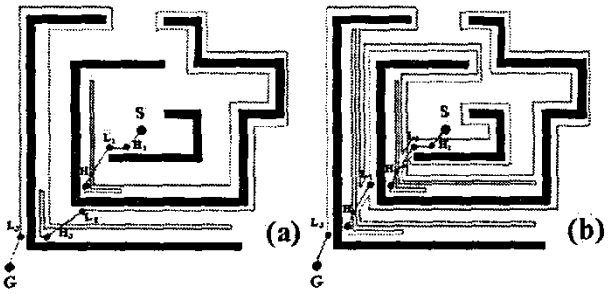

Fig. 13. (a) (a) A path selected in *Ave/wo* whose $\alpha = 0.5$ in a uncertain maze. (b) A path selected in *Ave/lo* whose $\alpha = 0.5$ in the maze.

is for a robot to select the most fruitful crack and direction on a certain graph spanned from a start point by the algorithm A^*. In general, avoiding larger intervals around an obstacle two times or less is more expensive than following smaller intervals around an obstacle three times or more. By this property, a new algorithm *Ave* always picks up a shorter path until a destination on the average.

REFERENCES

[1] A.Sankaranarayanan and M.Vidyasagar, "Path planning for moving a point object amidst unknown obstacles in a plane: The universal lower bound on worst case path lengths and a classification of algorithms," in *Proc. of the IEEE Int. Conf. on Robotics and Automation*, 1991, pp. 1734–1941.

[2] V.J.Lumelsky and A.A.Stepanov, "Path-planning strategies for a point mobile automaton moving amidst unknown obstacles of arbitrary shape."

[3] A.Sankaranarayanan and M.Vidyasager, "A new path planning algorithm for moving a point object amidst unknown obstacles in a plane," in *Proc. of the IEEE Int. Conf. on Robotics and Automation*, May 1990, pp. 1939–1936.

[4] A.Sankaranarayanan and M.Vidyasagar, "Path planning for moving a point object amidst unknown obstacles in a plane: a new algorithm and a general theory for algorithm development," in *Proc. of the Conf. on Decision and Control*, Dec. 1990, pp. 1111–1119.

[5] Y.Horiuchi and H.Noborio, "Evaluation of path length made in sensor-based path-planning with the alternative following," in *Proc. of the IEEE Int. Conf. on Robotics and Automation*, May 2001, pp. 1728–1735.

[6] R.Nogami, S.Hirao, and H.Noborio, "On the average path lengths of typical sensor-based path-planning algorithms by uncertain random mazes," in *Proc. of the IEEE Int. Symp. on Computational Intelligence in Robotics and Automation*, July 2003, pp. 471–478.

[7] H.Noborio and K.Urakawa, "A near-optimal sensor-based navigation among 2-d uncertain obstacles with simple shape," in *Proc. of the IEEE Int. Conf. on Robotics and Automation*, Apr. 1999, pp. 355–360.

[8] H.Noborio, Y.Maeda, and K.Urakawa, "Three or more dimensional sensor-based path-planning algorithm hd-i," in *Proc. of the IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, Oct. 1999, pp. 1699–1706.

[9] H.Noborio, K.Fujimiura, and Y.Horiuchi, "A comparative study of sensor-based path-planning algorithms in an unknown maze," in *Proc. of the IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, Oct. 2000, pp. 909–916.

[10] N.J.Nilsson, *Principles of artificial intelligence*. Tioga Publishing Co, 1980.

[11] V.J.Lumelsky and T.Skewis, "Incorporating range sensing in the robot navigation function," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 20, no. 5, pp. 1058–1068, 1990.

[12] I.Kamon, E.Rimon, and E.Rivlin, "Tangentbug: A range-sensor-based navigation algorithm," *Journal of Robotics Research*, vol. 17, no. 9, pp. 934–953, 1998.

[13] S.L.Laubach, *Theory and experiments in autonomous sensor-based motion planning with applications for flight planetary microrovers*. California Institute of Technology: Ph.D. thesis, 1999.

[14] S.L.Laubach and J.W.Burdick, "An autonomous sensorbased path-planner for planetary microrovers," in *Proc. of the IEEE Int. Conf. on Robotics and Automation*, May 1999, pp. 347–354.

[15] Y.H.Liu and S.Arimoto, "Path planning using a tangent graph for mobile robots among polygonal and curved obstacles," *Journal of Robotics Research*, vol. 11, no. 4, pp. 376–382, 1992.